



Ejercicios de Algoritmos Voraces

Ej. 1 — Se está celebrando una subasta para vender n objetos. Se reciben m pujas, cada una de la forma ‘quiero k_i objetos por e_i euros’ ($i = 1...m$). Caracterizar el problema como un problema de la mochila. ¿En qué condiciones es una mochila fraccional?

Ej. 2 — En una tienda hay una oferta. Si compras un producto, puedes llevarte gratis otro cualquiera de precio inferior o igual. Necesitas esta lista de productos: $a_1, a_2, \dots, a_n$, y quieres gastar lo mínimo posible. Diseña un algoritmo voraz que escriba en pantalla qué productos compras y cuáles te llevas gratis, y demuestra su corrección. Justifica su coste en tiempo.

Ej. 3 — Se nos proporciona una lista ordenada de n palabras. La longitud de la palabra que ocupa la posición i en la lista es w_i ; esto es, dicha palabra ocupa w_i posiciones (podemos suponer que no hay espacios entre las palabras para simplificar). Necesitamos romper la lista de palabras en líneas, sin cambiar su orden relativo.

La longitud de una línea es la suma de las longitudes de las palabras de esa línea. La longitud ideal de una línea es L . Ninguna línea puede ocupar más de L , pero sí que pueden ser más cortas. La penalización por tener una línea de longitud K es $L - K$. La penalización total para una distribución de palabras es la suma de las penalizaciones de cada línea. Debemos diseñar una distribución de palabras que minimice la penalización total.

Demostrar o encontrar un contraejemplo sobre si el siguiente algoritmo voraz resuelve el problema: “Colocar cada una de las palabras en la línea actual si cabe; si no, colocarla en la línea siguiente”

Ej. 4 — El problema de “calcular un subconjunto A ” de las aristas de un grafo con pesos, no dirigido y conexo G de forma que todos los vértices de G sigan estando conectados si se consideran únicamente las aristas de A y que la suma de los pesos de las aristas de A sea mínima” tiene sentido también aunque los pesos puedan ser negativos. Sin embargo, la solución puede no ser un árbol (como ocurre en el caso de pesos no negativos, con el árbol de recubrimiento de peso mínimo). Se trata de adaptar el algoritmo de Kruskal o el de Prim para que funcione también para grafos que puedan tener pesos negativos.

Ej. 5 — Demostrar que el algoritmo de Prim puede implementarse utilizando un cola con prioridades, al igual que el algoritmo de Kruskal. Demostrar que, con esa mejora, el algoritmo tiene un coste temporal en $\Theta(a \cdot \log n)$, donde a es el número de aristas del grafo y n es el número de vértices.

Ej. 6 — El profesor Sopmac quiere decorar el pasillo de su casa con cuadros. Como supone que sus visitantes se fijarán mejor en ellos si se encuentran próximos a las puertas, ha marcado en una recta las posiciones de las mismas.

Calcula que un cuadro está bien colocado si se encuentra a una distancia menor o igual de 0,75 metros de una puerta, y además piensa que cada una debería tener cerca un cuadro.

Lo que quiere es saber dónde colocar los cuadros de manera que se minimice el número que necesita. Para ello, debemos diseñar un algoritmo que encuentre una solución óptima.

Ej. 7 — Seguramente conoces un algoritmo de coste temporal $O(n+m)$ que mezcla dos ficheros ordenados de longitudes n y m , respectivamente, para crear un tercer fichero que contiene los datos de los dos y también está ordenado (Ojo. La experiencia dice que poca gente lo programa bien a la primera). Cuando se trata de mezclar más de dos ficheros ordenados, puede conseguirse mediante llamadas sucesivas al mismo algoritmo, ordenando ficheros por pares. Así, si los ficheros ordenados F_1 , F_2 , F_3 y F_4 deben mezclarse en uno solo, puede conseguirse mezclando primero F_1 y F_2 , para obtener F_{12} . A continuación pueden mezclarse F_{12} y F_3 , para obtener F_{123} . Por último, mezclando F_{123} y F_4 , se obtiene el fichero deseado. Pero no es ésta la única solución. Podríamos haber mezclado primero F_1 y F_2 , para obtener F_{12} . Después, F_3 y F_4 , para obtener F_{34} . Y, por último, mezclar F_{12} y F_{34} , para obtener el mismo fichero final. Dados n ficheros, hay muchas formas de mezclarlos por parejas para obtener el mismo fichero final. Las diferentes formas de emparejar los ficheros en las mezclas pueden requerir distintos tiempos de ejecución. Diseñar un algoritmo que calcule la forma óptima (i.e., con menos comparaciones entre elementos) de mezclar n ficheros cuyas longitudes son conocidas, mediante llamadas sucesivas al algoritmo de mezcla de una pareja de ficheros y justificar su corrección.

Ej. 8 — El profesor Allilev conduce su automóvil desde Malanquilla hasta Zaragoza por la comarcal 1313. El depósito de su automóvil tiene capacidad para recorrer n kilómetros sin repostar (a la velocidad habitual de su marcha). El profesor Allilev siempre lleva consigo el mapa de carreteras en el que están reflejadas las distancias entre las estaciones de servicio que va a encontrar en su camino. Para ganar tiempo, nuestro querido profesor Allilev desea hacer el mínimo número de paradas que sea posible para repostar combustible. Diseñar un algoritmo eficiente que indique al profesor en qué estaciones debe repostar y justificar su corrección.

Ej. 9 — Suponer que en un problema de la mochila 0-1 (es decir, con objetos no fraccionables) la ordenación de los objetos si se ordenan por pesos crecientes es la misma que si se ordenan por beneficios decrecientes. Diseñar un algoritmo (voraz) eficiente para encontrar la solución óptima y justificar su corrección.

Ej. 10 — Se dice que un punto p del plano euclídeo domina a otro q cuando las coordenadas de p son mayores o iguales que las de q ($p_x \geq q_x$ y $p_y \geq q_y$) y además $p \neq q$. Se dice que un punto p es maximal en un conjunto de puntos C si no existe ningún punto q perteneciente a C que domine a p . Diseñar un algoritmo que, dado un conjunto C de puntos en el plano, calcule el conjunto de todos los puntos maximales de C .

Ej. 11 — Sean dos secuencias de n números naturales cada una, $A = (a_1, a_2, \dots, a_n)$ y $B = (b_1, b_2, \dots, b_n)$, la primera de las cuales, A , está ordenada en forma creciente. Se trata de diseñar un algoritmo que obtenga la reordenación de la secuencia B de forma que maximice la suma de los elementos de A que no superan al elemento correspondiente de B ; es decir: maximizar $\sum_{i|a_i \leq b_i} a_i$

Ejemplo. Sean $A=(2,6,7,10,22)$ y $B=(15,1,7,5,2)$. La reordenación óptima de B es $B'=(5,2,7,15,1)$, con un beneficio de $2+7+10=19$.

Ej. 12 — Considerar la siguiente estrategia voraz para el cálculo del árbol de recubrimiento de diámetro mínimo:

- Sea la solución inicial un árbol consistente en un vértice arbitrario y ninguna arista.
- Mientras que los vértices restantes no se hayan incluido en el árbol, seleccionar la arista que conecta algún vértice del árbol con algún vértice que no lo está y tal que el incremento del diámetro del árbol sea mínimo.

¿Es cierto que este algoritmo calcula un árbol de recubrimiento de diámetro mínimo? Justificar la respuesta.

Ej. 13 — Un monitor de esquí dispone de n pares de esquís para asignar a sus n alumnos novatos. De acuerdo con las normas de la escuela, cada alumno debería recibir un par de esquís adecuados para su estatura. El problema del monitor es asignar los n pares a sus n alumnos minimizando la suma de los valores absolutos de las diferencias entre la altura de cada alumno y la longitud del par de esquís asignado.

El problema puede resolverse de forma óptima con el método siguiente: asignar el par de esquís más cortos al alumno más bajo y proceder de la misma forma en orden creciente de longitudes y alturas. Demostrar la corrección de esta solución. (Nota: Esta solución también minimiza la mayor diferencia entre altura y longitud.)

Ej. 14 — El algoritmo descrito en el ejercicio anterior puede considerarse un algoritmo voraz; sin embargo, no es voraz con respecto a la función de selección obvia (es decir, su función de selección no coincide con la función objetivo). Describir la estructura de matroide subyacente debajo del problema. Explicar también por qué el algoritmo voraz no funcionaría si se utilizase como función de selección otra más obvia basada en la función objetivo.

Ej. 15 — El algoritmo GeoGreedy de enrutado voraz geográfico para alcanzar un nodo objetivo, o desde otro nodo inicial i consiste en recorrer un grafo de la siguiente forma:

- Si un mensaje m llega al nodo u elige de sus nodos adyacentes el que está geográficamente más cercano al objetivo o .
- Si u es el más próximo al objetivo, se lo entrega y termina el proceso.

Comentar **brevemente** la estructura de datos adecuada para resolver este problema. ¿Encuentra esta estrategia voraz la solución óptima? Demostrarlo si la respuesta es si, o mostrar un contraejemplo en caso de que sea falso.

Ej. 16 — Debemos hacer un recorrido entre dos ciudades en un coche eléctrico, en el menor tiempo posible. El vehículo tiene una autonomía de x kilómetros después de recargarlo completamente (en las condiciones habituales de ese recorrido). El sistema de ayuda del vehículo dispone de la información sobre los cargadores, con las distancias entre los mismos a lo largo del recorrido. Teniendo en cuenta que el tiempo de carga no es cero, se desea hacer el menor número posible de paradas.

Se pide:

- Indicar la estrategia algorítmica elegida.
- Diseñar un algoritmo eficiente que indique en qué estaciones de carga se deben planificar las paradas.
- Indicar el coste del algoritmo.

Ej. 17 — Cualquier fracción positiva se puede representar como la suma de fracciones unitarias. Una fracción es unitaria si el numerador es 1 y el denominador es un entero positivo. Por ejemplo, $1/5$ es una fracción unitaria. Esta representación se denomina **Fracción Egipcia** porque la utilizaban los antiguos egipcios.

Algunos ejemplos. La representación en formato de fracciones egipcias de:

- $2/3$ es $1/2 + 1/6$
- $6/14$ es $1/3 + 1/11 + 1/231$
- $12/13$ es $1/2 + 1/3 + 1/12 + 1/156$

Encontrar un algoritmo voraz para obtener la representación de una fracción cualquiera n/m , con $m > n$.

Ej. 18 — Evaluar y mejorar las páginas de la Wikipedia hispana sobre este tema (o alguno relacionado) (<http://es.wikipedia.org/>) (y el resultado final publicado allí), contrastándolo con la bibliografía y con las páginas equivalentes de la Wikipedia inglesa (<http://en.wikipedia.org/>). Naturalmente, esta última también es posible que pueda mejorarse.

Ej. 19 — Comprobar qué información nos ofrece alguna de las inteligencias artificiales sobre los siguientes temas, en español y en inglés:

- Algoritmia (*algorithmics*)
- Algoritmos voraces (*greedy algorithms*)

¿Proporciona respuestas? ¿Son correctas y adecuadas? ¿Se podrían mejorar?

Ej. 20 — Dado n escribir el máximo número de números compuestos (números que no son primos) que suman n (recordar que los primeros números compuestos son: 4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, ...). Ejemplos:

- Entrada: 90. Salida: 22
 $90 = 21 * 4 + 6$

Observación: el 4 aparece 21 veces, se repite tantas como sea posible/necesario.

- Entrada: 10. Salida: 2
 $10 = 4 + 6$

- Entrada: 29. Salida: 6
 $29 = 4 * 5 + 9$

- Entrada: 35. Salida: 7
 $35 = 4 * 5 + 6 + 9$

Notas:

1. Si el número es menor que 4 no habrá ninguna combinación.
2. Si el número es 5, 7, 11 no habrá ninguna división posible.

¿Es posible diseñar un algoritmo voraz para calcular la descomposición para números mayores o iguales que 4 que no sean 5, 7, 11?

Sugerencia: puesto que lo mejor es poner tantos 4's como sea posible, el resto de dividir el número por 4 nos ayudará a determinar qué número compuesto podemos usar para obtener la solución.

Ej. 21 — Tenemos una pared de longitud L y suficientes estanterías para colgar de dos tipos de longitud $l_1, l_2 \leq L$ ($l_1 \neq l_2$); las más largas son más baratas.

El objetivo es poner estanterías en la pared de forma que se minimice la longitud que queda libre y, en caso de soluciones con la misma longitud libre, preferimos las más baratas.

Ejemplos:

Entrada	Salida	Comentarios
$L = 24, l_1 = 3, l_2 = 5$	3 3 0	Tres unidades de cada tipo y quedará 0 espacio libre. $3 \cdot 3 + 3 \cdot 5 = 24$. Otra solución podría haber sido 8 0 0, pero como las estanterías más largas son más baratas la solución 3 3 0 es mejor.
$L = 29, l_1 = 3, l_2 = 9$	0 3 2	0 unidades de l_1 , 3 unidades de l_2 y quedan 2 unidades de medida sin estantería. $0 \cdot 3 + 3 \cdot 9 + 2 = 29$
$L = 24, l_1 = 4, l_2 = 7$	6 0 0	Con 6 unidades de l_1 dejamos la pared cubierta. $6 \cdot 4 + 0 \cdot 7 + 0 = 24$

Se pide:

- Diseñar una solución eficiente mediante el esquema voraz.
- Hacer una estimación del coste en tiempo de la solución.

Ej. 22 — Dados dos vectores *llegadas* y *salidas* que representan los horarios de llegada y salida en una estación de trenes, respectivamente, necesitamos encontrar el mínimo número de andenes necesarios para que ningún tren tenga que esperar.

Ejemplo ₁	t_1	t_2	t_3	t_4	t_5	t_6	Ejemplo ₂	t_1	t_2
llegadas	9:00	9:40	9:50	11:00	15:00	18:00	llegadas	1:00	5:00
salidas	9:10	12:00	11:20	11:30	19:00	20:00	salidas	3:00	7:00

Resultados:

Ejemplo₁. 3 andenes (t_2 , t_3 y t_4 llegan entre las 9:40 y las 11:00 y salen desde las 11:20).

Ejemplo₂. 1 anden

Se pide:

- Desarrollar un algoritmo que implemente una solución eficiente para este problema.

Ej. 23 — Tenemos que repartir galletas a unos niños sabiendo el tamaño mínimo de la galleta que quiere cada uno. Para ello, tenemos un vector D donde la posición i indica el tamaño mínimo de galleta que quiere el niño i . Además tenemos otro vector G , donde la posición j representa el tamaño de la galleta j . Queremos saber el máximo número de niños a los que podemos satisfacer con las galletas disponibles (le daremos a cada uno una galleta como máximo).

Un niño i estará satisfecho con la galleta j si es del tamaño elegido o una más grande. Esto es, $D[i] \leq G[j]$

Ejemplos:

- Si $D = [1, 2, 3]$ y $G = [1, 1]$ sólo podemos satisfacer a un niño.
- si $D = [1, 2]$ y $G = [1, 2, 3]$ podremos satisfacer a los dos niños.
- si $D = [3, 1, 4]$ y $G = [1, 4, 3]$ podremos satisfacer a los tres niños.

Se pide:

Desarrollar un algoritmo que implemente una solución eficiente para este problema; analizar el coste de la solución.